

Campus Navigation Code Explanation

This document explains the Campus Navigation Android project folder by folder. The app is a Java Android application for campus navigation. It uses Firebase Authentication, Firestore, Firebase Cloud Messaging, Room database, WorkManager, Google Maps, location services, and a simulated indoor beacon navigation system.

The project follows an MVVM-style structure: UI screens call ViewModels, ViewModels call Repositories, Repositories load data from Firebase, Room, preferences, or bundled asset JSON files, and the UI observes the results.

Root Project Folder

settings.gradle names the project and includes the app module. This tells Gradle that the Android application lives in the app folder.

build.gradle at the root level defines project-wide Gradle plugins, including the Android Gradle plugin and Google Services plugin used by Firebase.

gradle.properties stores build configuration such as AndroidX support, Jetifier, Gradle parallel execution, and JVM memory settings.

gradlew and gradlew.bat are Gradle wrapper launchers. They allow the project to build with the configured Gradle version without requiring a manually installed Gradle.

README.md is the setup and feature guide. It explains Firebase setup, Maps API setup, Firestore collections, and the main app features.

.gitignore prevents generated files, local machine files, and secrets such as google-services.json from being committed.

app Folder

The app folder is the Android application module. It contains the Gradle configuration, manifest, Java source code, XML resources, and bundled asset files.

app/build.gradle defines the package com.example.campusnavigation, SDK versions, Java version, View Binding, BuildConfig, and all app dependencies. Important dependencies include AppCompat, Material Components, Lifecycle ViewModel and LiveData, Room, WorkManager, Google Maps, Google Location Services, Firebase Auth, Firestore, FCM, Gson, and Guava.

app/proguard-rules.pro is reserved for release code shrinking rules. Since minification is disabled, it currently acts mostly as a placeholder.

AndroidManifest.xml

The manifest declares the app package, permissions, activities, services, and Google Maps metadata. It requests internet, network, location, notification, and Bluetooth permissions.

SplashActivity is the launcher activity. AuthActivity handles login and registration. MainActivity is the main screen after authentication. CampusMessagingService receives Firebase Cloud Messaging notifications.

The manifest also registers CampusNavigationApp as the custom Application class. This class is created before activities and is used for app-wide initialization.

Application Bootstrap

CampusNavigationApp.java initializes core dependencies when the app starts. It creates or exposes the Room database, repositories, preferences repository, notification channels, and saved theme mode.

This class acts as a simple dependency provider. ViewModels can access repositories from the application instance instead of constructing them independently.

model Folder

The model folder contains plain Java objects that describe app data.

Building.java represents one campus building. It stores an id, name, description, type, latitude, longitude, and whether it is a favorite. The Firestore document id is mapped onto the building id.

CampusEvent.java represents a campus event. It includes event id, event name, building id, building name, event time in milliseconds, and description.

BeaconNode.java represents a simulated indoor beacon. It contains an id, room name, x and y coordinates for the indoor map, and a simulated signal value.

IndoorRoute.java represents a generated indoor route. It stores the starting room, ending room, and a list of instruction steps.

data/local Folder

The data/local folder is the Room database layer. Room is an Android library that wraps SQLite with Java annotations and DAO interfaces.

AppDatabase.java defines the local database named campus_navigation.db. It registers the BuildingEntity and EventEntity tables and exposes DAOs for database access.

BuildingEntity.java defines the buildings table. It is the database form of a Building object and contains fields that Room stores in SQLite.

EventEntity.java defines the events table. It is the database form of a CampusEvent object.

BuildingDao.java contains database operations for buildings, including observing all buildings, searching, loading favorites, inserting data, updating favorite state, and counting stored buildings.

EventDao.java contains database operations for events, including observing all events, inserting event lists, counting events, and loading stored events.

data/mapper Folder

Mapper classes convert between database entities and UI/domain models. This keeps the database format separate from the rest of the app.

BuildingMapper.java converts BuildingEntity objects into Building objects and converts Building objects back into BuildingEntity objects before saving.

EventMapper.java does the same conversion work for EventEntity and CampusEvent.

data/repository Folder

Repositories coordinate where data comes from and where it is saved. They hide Firebase, Room, SharedPreferences, and asset-loading details from the UI.

BuildingRepository.java manages building data. It syncs buildings from Firestore into Room, loads sample buildings from assets when needed, exposes searchable LiveData, provides autocomplete suggestions, and toggles favorites.

EventRepository.java manages event data. It syncs events from Firestore into Room and uses WorkManager to schedule reminder notifications before future events.

AuthRepository.java wraps Firebase Authentication. It supports email login, registration, anonymous guest sign-in, sign-out, and checking whether a user is logged in.

PreferencesRepository.java wraps SharedPreferences. It stores dark mode, notification preference, and language choice.

viewmodel Folder

ViewModels hold screen state and expose data to Activities and Fragments through LiveData. They make the UI easier to test and keep business logic out of XML and screen classes.

AuthViewModel.java exposes login, register, and guest login operations. It wraps results in a Resource object so the UI can show loading, success, or error states.

BuildingViewModel.java exposes building lists, search queries, sync actions, suggestions, and favorite updates.

EventViewModel.java exposes event lists and calculates active events for map display.

SettingsViewModel.java exposes user settings and logout behavior.

IndoorNavigationViewModel.java runs the simulated beacon scan loop, updates current room state, and generates indoor route steps.

ui Folder

The ui folder contains Activities, Fragments, adapters, and small UI contracts.

ui/splash/SplashActivity.java shows a short splash screen, then checks authentication. Logged-in users go to MainActivity. Other users go to AuthActivity.

ui/auth/AuthActivity.java displays the login and registration form. It validates user input, supports guest login, calls AuthViewModel, and reacts to success or error states.

ui/main/MainActivity.java is the main container. It hosts the bottom navigation, switches between dashboard, map, directory, events, and settings fragments, and requests runtime permissions.

ui/common/NavigationHost.java is an interface used by fragments when they need MainActivity to perform navigation actions, such as opening the map for a selected building.

UI Fragments

DashboardFragment.java is the home tab. It shows summary information, highlight cards, and entry points such as indoor navigation.

MapFragment.java is the largest screen. It displays Google Maps, building markers, event markers, search suggestions, type filters, a building detail bottom sheet, route drawing, live location tracking, ETA display, turn-by-turn style instructions, and rerouting.

DirectoryFragment.java shows a searchable RecyclerView of buildings. Tapping a building opens it on the map. Long-pressing can toggle favorite state.

EventsFragment.java shows campus events in a list.

SettingsFragment.java contains switches and controls for notifications, dark mode, language, manual sync, and logout.

IndoorNavigationFragment.java displays the simulated indoor navigation screen. It shows beacon status, room input fields, route steps, and an animated blue dot on an indoor grid.

ui/adapter Folder

Adapters connect RecyclerView data to XML row layouts.

BuildingAdapter.java renders each building row in the directory.

EventAdapter.java renders each campus event row.

HighlightAdapter.java renders dashboard highlight cards.

util Folder

Resource.java is a generic loading wrapper. It lets ViewModels report SUCCESS, ERROR, or LOADING with optional data and messages.

AppExecutors.java provides background executors so database and network work do not block the main UI thread.

SampleDataLoader.java reads bundled JSON files from the assets folder and converts them with Gson.

PermissionHelper.java centralizes permission checks and request arrays for location, Bluetooth, and notifications.

LocationHelper.java uses the fused location provider to get the current GPS position and location updates.

RouteProvider.java is the routing coordinator. It first tries Google Directions API and falls back to local graph routing when needed.

DirectionsRouteFetcher.java performs the HTTP request to Google Directions API using the Maps API key from BuildConfig.

CampusRouteCalculator.java loads campus_graph.json and calculates routes on a local graph. This allows navigation to still work when online directions are unavailable.

IndoorNavigationSimulator.java simulates BLE beacon behavior with hardcoded beacon nodes and generated indoor route instructions.

NotificationHelper.java creates notification channels and displays event reminder or push notification messages.

service and worker Folders

service/CampusMessagingService.java handles Firebase Cloud Messaging. When a push message arrives, it creates a local notification through NotificationHelper.

worker/EventReminderWorker.java is used by WorkManager. It runs at scheduled times and shows a reminder notification before a campus event.

assets Folder

app/src/main/assets/sample/buildings.json contains bundled sample building data. The app can load this when Firestore is empty or unavailable.

app/src/main/assets/sample/events.json contains bundled sample event data.

app/src/main/assets/campus_graph.json contains nodes and edges for local route calculation. CampusRouteCalculator uses this file for offline pathfinding.

res Folder

res/layout contains XML layouts for each screen and RecyclerView item. Examples include activity_splash.xml, activity_auth.xml, activity_main.xml, fragment_dashboard.xml, fragment_map.xml, fragment_directory.xml, fragment_events.xml, fragment_settings.xml, fragment_indoor_navigation.xml, item_building.xml, item_event.xml, and item_highlight.xml.

res/values/strings.xml stores English UI text. res/values-am/strings.xml stores Amharic translations.

res/values/colors.xml defines light theme colors. res/values-night/colors.xml defines dark theme overrides.

res/values/themes.xml defines the Material DayNight app theme and splash theme.

res/drawable contains XML drawables and icons such as the splash background, map bottom sheet background, indoor blue dot, and launcher foreground.

res/menu/bottom_nav_menu.xml defines the bottom navigation items: dashboard, map, directory, events, and settings.

res/xml contains Android backup and data extraction rule files.

End-to-End App Flow

When the app launches, SplashActivity starts first. It waits briefly and checks Firebase authentication state. If a user is already logged in, MainActivity opens. Otherwise, AuthActivity opens.

AuthActivity lets the user sign in, register, or continue as a guest. After successful authentication, the app opens MainActivity.

MainActivity requests needed runtime permissions and displays bottom navigation. Each tab loads a Fragment.

Dashboard, Directory, Map, and Events use ViewModels. The ViewModels ask repositories to sync data. Repositories load Firestore data into Room. The UI observes Room LiveData so screens can update automatically and work offline after caching.

For outdoor navigation, MapFragment asks RouteProvider for a route. RouteProvider tries Google Directions first. If that fails, CampusRouteCalculator uses campus_graph.json to calculate a local walking route.

For indoor navigation, IndoorNavigationViewModel uses IndoorNavigationSimulator. It does not use real BLE hardware yet. It cycles through fake beacons and generates demonstration route steps.

For events, EventRepository syncs events, saves them to Room, and schedules reminder jobs. EventReminderWorker later displays notifications. Firebase push messages are handled by

CampusMessagingService.

Important Setup Notes

The project needs `app/google-services.json` for Firebase. This file is normally downloaded from the Firebase console and is intentionally ignored by git.

The Maps API key in `app/build.gradle` should be replaced with a real key for Google Maps and Directions API features.

Indoor navigation is currently simulated. Real BLE navigation would require scanning real beacons, estimating distance, and mapping beacon positions to indoor coordinates.

There are no visible unit or instrumentation test folders in the current structure.